

# AWS - DynamoDB, CodePipeline y CodeDeploy

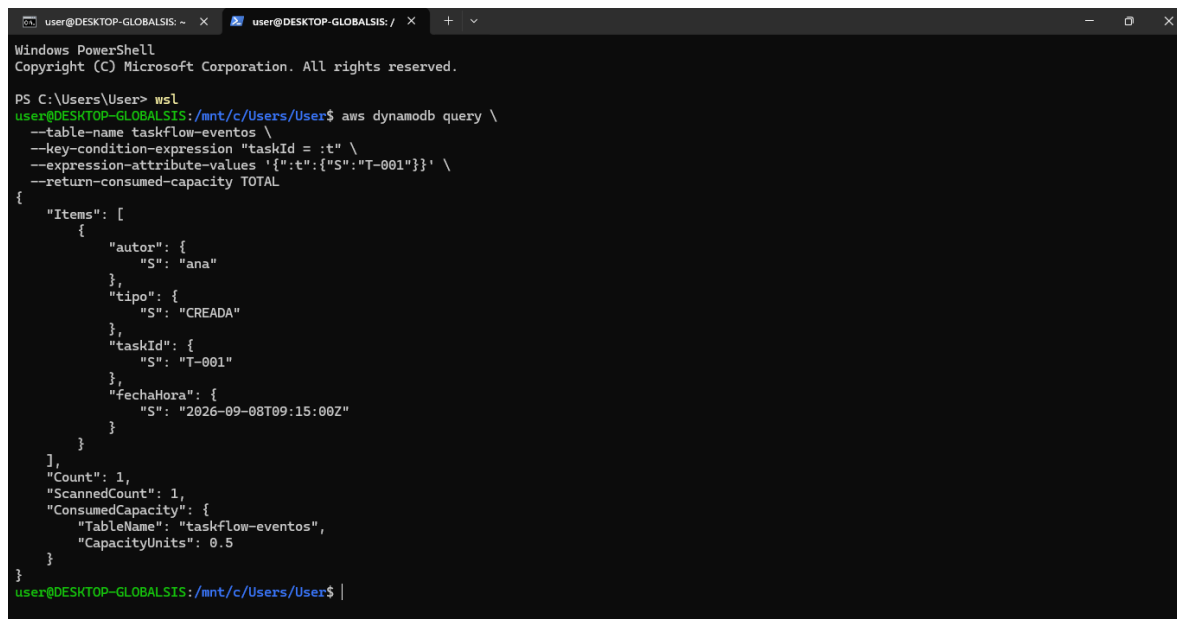
## Amazon DynamoDB (Bases de Datos NoSQL y CLI)

### ¿Qué es el concepto?

DynamoDB es una base de datos NoSQL gestionada por AWS que utiliza un modelo de clave-valor con un esquema flexible, usando una escala horizontal sin límite. Lo diferente con las bases relacionales, aquí el modelado se hace al revés: primero definimos los patrones de acceso o preguntas que hará la aplicación, y con eso se diseña la tabla. Los componentes principales para buscar datos son la Partition Key (decide en qué servidor físico vive el dato para repartir la carga) y la Sort Key (que ordena los elementos dentro de esa partición).

### El proceso que realicé:

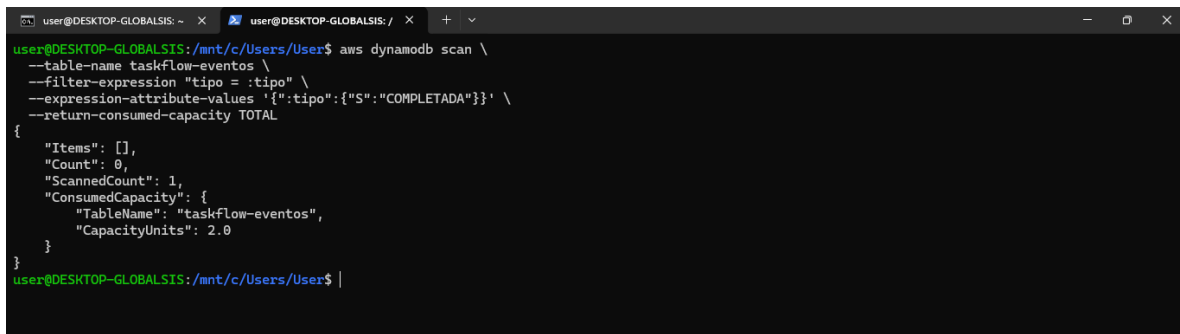
Primero configuré la AWS CLI en mi terminal usando una clave de acceso generada para mi usuario taskflow-admin mediante el comando `aws configure`. Luego, creé la tabla `taskflow-eventos` desde la terminal definiendo `taskId` como Partition Key (tipo HASH) y `fechaHora` como Sort Key (tipo RANGE), utilizando el modo de facturación bajo demanda (PAY\_PER\_REQUEST). Después inserté cinco registros de prueba en formato JSON (especificando los tipos de dato, usando el comando `put-item`). Y ya al final, comparé el rendimiento y costo leyendo datos haciendo un query específico por `taskId`, AWS me cobró 0.5 unidades de capacidad (CapacityUnits), pero al usar `scan` con un filtro para buscar tareas completadas, consumió 2.0 unidades.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\User> wsl
user@DESKTOP-GLOBALSIS:/mnt/c/Users/User$ aws dynamodb query \
--table-name taskflow-eventos \
--key-condition-expression "taskId = :t" \
--expression-attribute-values '{"t":{"S":"T-001"}}' \
--return-consumed-capacity TOTAL
{
  "Items": [
    {
      "autor": {
        "S": "ana"
      },
      "tipo": {
        "S": "CREADA"
      },
      "taskId": {
        "S": "T-001"
      },
      "fechaHora": {
        "S": "2026-09-08T09:15:00Z"
      }
    }
  ],
  "Count": 1,
  "ScannedCount": 1,
  "ConsumedCapacity": {
    "TableName": "taskflow-eventos",
    "CapacityUnits": 0.5
  }
}
```

*Ilustración 1 - Query 0.5 CapacityUnits*

A terminal window with a dark background. The prompt is 'user@DESKTOP-GLOBALSIS: /mnt/c/Users/User\$'. The command entered is 'aws dynamodb scan \ --table-name taskflow-eventos \ --filter-expression "tipo = :tipo" \ --expression-attribute-values '{"S": "COMPLETADA"}' \ --return-consumed-capacity TOTAL'. The output is a JSON object: {'Items': [], 'Count': 0, 'ScannedCount': 1, 'ConsumedCapacity': {'TableName': 'taskflow-eventos', 'CapacityUnits': 2.0}}.

```
user@DESKTOP-GLOBALSIS: /mnt/c/Users/User$ aws dynamodb scan \
--table-name taskflow-eventos \
--filter-expression "tipo = :tipo" \
--expression-attribute-values '{"S": "COMPLETADA"}' \
--return-consumed-capacity TOTAL
{
  "Items": [],
  "Count": 0,
  "ScannedCount": 1,
  "ConsumedCapacity": {
    "TableName": "taskflow-eventos",
    "CapacityUnits": 2.0
  }
}
user@DESKTOP-GLOBALSIS: /mnt/c/Users/User$
```

## *Ilustración 2 - Scan 2.0 CapacityUnits*

### **Comentarios personales:**

El experimento de comparar query contra scan me abrió los ojos. Descubrí que un scan con filtro no te ahorra nada de dinero ni recursos porque DynamoDB nos lee la tabla entera (y te la cobra completa) antes de descartar lo que no coincide con el filtro, lo bueno que no eran tantos registros. Entendí por qué en DynamoDB, si necesitas hacer consultas complejas o JOINS, te equivocaste de base de datos.

### **Respondiendo a la pregunta del proyecto: ¿Qué midió tu comparación de query contra scan?**

Midió el consumo real de recursos o CapacityUnits (lo que AWS nos cobra). Confirmé que un query va directo a la partición por medio de su clave y cuesta casi nada (0.5 unidades). En cambio, un scan con filtro es parece hasta estafa porque DynamoDB lee todos los registros de la tabla entera y te los cobra todos (2.0 unidades en mi prueba de 5 registros), para después tirar la basura que no coincide con el filtro. En una tabla de millones de datos, un scan sería muy caro.

## Preparación de Infraestructura (EC2, S3 y Agente de CodeDeploy)

## ¿Qué es el concepto?

Para que un pipeline de despliegue continuo funcione, la infraestructura debe estar preparada para comunicarse sin intervención humana. La instancia EC2 necesita un perfil de IAM para tener permiso de descargar archivos desde S3, y una etiqueta (Tag) específica para que CodeDeploy sepa en qué máquina instalar la aplicación. El bucket de S3 que almacena el código compilado debe tener el versionado activado para ser compatible con CodePipeline, y la EC2 debe tener instalado el "agente" de CodeDeploy, que sería el proceso en segundo plano que espera órdenes de despliegue.

### El proceso que realicé:

Relancé mi EC2 agregando el perfil de instancia `taskflow-ec2-role` (con permisos `AmazonS3ReadOnlyAccess`) y le asigné la etiqueta `Name = taskflow-ec2`. Una vez creada, entré por SSH e instalé Java 21 (`java-21-amazon-corretto-headless`). Después, creé un nuevo bucket en S3 llamado `taskflow-artefactos-inzunza` habilitando la opción "Bucket Versioning". Por último, dentro de la EC2, instalé Ruby y utilicé `wget` para descargar e instalar el agente de CodeDeploy, comprobando que estuviera activo con `systemctl status codedeploy-agent`.

[illegible]

### Ilustración 3 - Active running del servicio codedeploy-agent

**Comentarios personales:**

Me di cuenta de que olvidarnos de algo en esta fase arruina todo el pipeline. Si olvidamos el Tag, CodeDeploy nos da "éxito" pero no despliega nada porque no encuentra la máquina. Si olvidamos el rol de IAM, el agente falla al intentar bajar el .jar del bucket.

## Archivos de Configuración del Pipeline (buildspec, appspec y hooks)

### ¿Qué es el concepto?

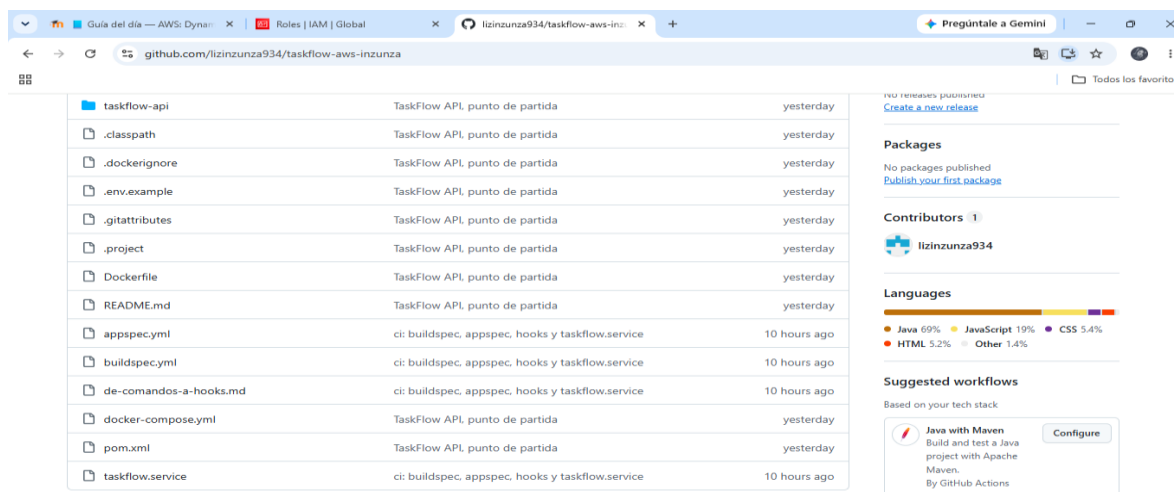
Para automatizar el despliegue, los comandos manuales que usábamos antes se traducen a archivos de configuración versionados en el repositorio. `buildspec.yml` instruye a CodeBuild sobre cómo compilar el proyecto. `appspec.yml` le dice al agente de CodeDeploy qué archivos copiar y qué scripts ejecutar en cada hooks del ciclo de vida. Un archivo `.service` permite usar `systemd` en Linux para que la API se mantenga viva en segundo plano y se reinicie sola si falla.

### El proceso que realicé:

Copie cuatro componentes clave en mi repositorio:

- `taskflow.service`: Está configurado para que ejecutara el `.jar` con el usuario `ec2-user`, en el directorio `/opt/taskflow`, y añadí `Restart=always` para garantizar su disponibilidad.
- `buildspec.yml`: Está configurado para definir el entorno con `corretto21`.
- `appspec.yml`: Está configurado para mapear los 4 hooks principales a mis scripts de bash: `ApplicationStop` (`parar.sh`), `AfterInstall` (`permisos.sh`), `ApplicationStart` (`arrancar.sh`) y `ValidateService` (`verificar.sh`).
- Permisos en Git: Utilicé el comando `git update-index --chmod=+x scripts/*.sh` para que los scripts fueran ejecutables al descargarse en la EC2, e hice el push a GitHub. Todo este código y configuración quedó almacenado y versionado en mi repositorio de GitHub, el cual puede ser consultado aquí:

<https://github.com/lizinzunza934/taskflow-aws-inzunza>



***Ilustración 4 - Archivos `appspec.yml`, `buildspec.yml` y `taskflow.service` pusheados al repositorio a la rama principal.***

## **Comentarios personales:**

Fue bueno ver cómo las tareas manuales que hice el día anterior se convirtieron en código estructurado. Entender que `ApplicationStop` incluye un `|| true` para no fallar en el primer despliegue (cuando no hay nada que detener) o que `ValidateService` hace peticiones repetidas al endpoint `/info` en lugar de usar un simple "sleep", me enseñó cómo hacer scripts tolerantes.

## **Respondiendo a la pregunta del proyecto: ¿Qué hace cada pieza del pipeline y por qué el `appspec.yml` viaja dentro del artefacto?**

- `CodePipeline` es el director de orquesta que arranca todo cuando detecta un cambio en GitHub.
- `CodeBuild` lee mi `buildspec.yml` para saber cómo compilar el código Java y empaquetarlo en un `.jar`.
- `S3` sirve como el puente o almacén seguro donde `CodeBuild` guarda el ZIP terminado.
- `CodeDeploy` es quien toma el ZIP, busca mi EC2, y lee el `appspec.yml` para saber cómo instalarlo. El `appspec.yml` viaja dentro del ZIP (artefacto) porque `CodeDeploy` no revisa mi GitHub solo descarga el ZIP desde `S3`, así que las instrucciones de cómo instalar la aplicación deben venir empaquetadas junto al `.jar`.

## **Respondiendo a la pregunta del proyecto: ¿En qué hook vive cada comando que corriste a mano? Todo lo que hacía manual se automatizó en estos hooks:**

- Matar el proceso (`kill <PID>`) ahora vive en `ApplicationStop` ejecutando `parar.sh`.
- Cambiar permisos (`chown /opt/taskflow`) ahora vive en `AfterInstall` ejecutando `permisos.sh`.
- Arrancar la API (`nohup java -jar...`) ahora vive en `ApplicationStart` ejecutando `arrancar.sh` (que levanta el servicio de `systemd`).
- Abrir el navegador para ver si contestaba ahora vive en `ValidateService` ejecutando `verificar.sh` (haciendo un bucle de peticiones `curl`).

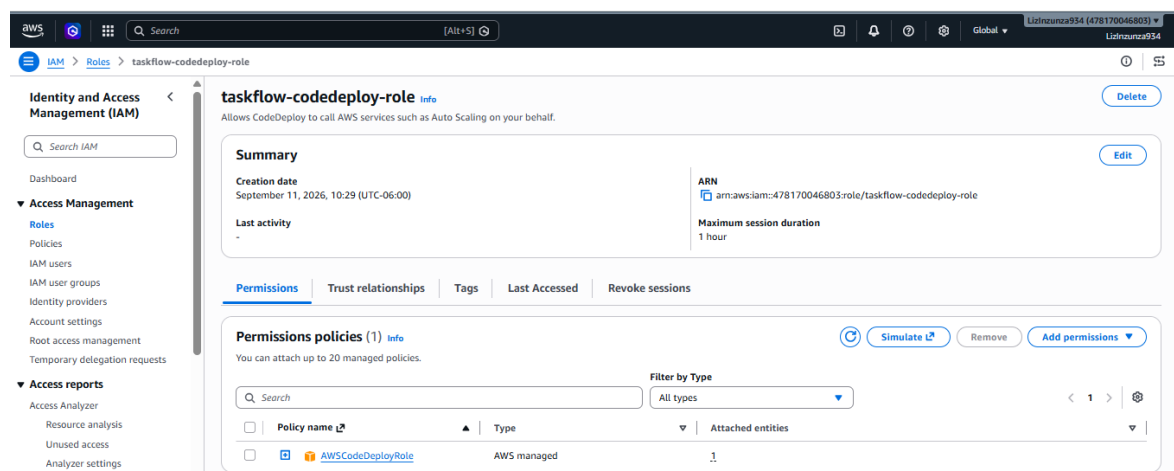
## IAM Roles y Pausa de la Práctica en CodeDeploy

### ¿Qué es el concepto?

En AWS, los servicios necesitan permisos para hablar entre ellos, también los usuarios. Para que CodeDeploy pueda interactuar con las instancias EC2 y gestionar los despliegues, necesitamos un rol de servicio configurado previamente.

### El proceso que realicé:

Llegué hasta el Paso 9 de la guía, donde comencé a unir los servicios. Fui a la consola de IAM, seleccioné la entidad de confianza de CodeDeploy y creé un rol llamado `taskflow-codedeploy-role`, el cual incluyó la política `AWSCodeDeployRole`. En este punto de la práctica no seguí, ya que el sistema requería reconfigurar el método de pago de la cuenta para poder continuar con la creación del pipeline en CodePipeline y la ejecución de CodeDeploy.



*Ilustración 5 - AWS IAM mostrando el rol de taskflow-codedeploy-role creado con éxito*

### Comentarios personales:

Aunque no pude ver el "push final" activando automáticamente la cadena de despliegue por la restricción de pago, preparar el terreno me dejó clara la arquitectura de CI/CD. Entendí cómo CodeBuild lee GitHub, empaqueta el artefacto en S3, y cómo CodeDeploy utiliza ese artefacto para ejecutar mis scripts dentro de la EC2 sin que yo toque la consola.